Atividade 4: ICMPv4 e ICMPv6

Nome: Matheus Milanezi

1. Caracterização do ambiente de experimentação

- Cenário A (rede local controlada):

Sistema operacional (host): Windows 10 Pro, Docker Desktop (backend WSL2)

Virtualização: containers Docker (Ubuntu 24.04) via Docker Compose, 4 hosts (c1, r1, r2, c2)

Ferramenta de captura: tcpdump -i any (roda dentro do container r1, capturando as duas interfaces dele de uma vez)

- Cenário B (Internet real):

Sistema operacional: Windows 10 Pro, direto no Windows.

Interface de rede real: vEthernet (Hyper-V) 2, um Hyper-V External Switch em modo bridge, criado para permitir IP fixo em VMs Hyper-V. Esse switch retira a pilha IP da placa física (que passa a aparecer como "Não Conectado" no Windows, mesmo trafegando tráfego real) e transfere pra esse adaptador virtual.

Endereço IPv4: 192.168.1.2/24, gateway 192.168.1.254

Endereço IPv6: 2804:d51:4764:7c00:65b:4891:a198:f7bf (global, prefixo alocado pelo LACNIC), com endereço temporário de privacidade (RFC 4941). Gateway IPv6 fe80::1 (link-local do roteador doméstico)

Conectividade IPv6 nativa confirmada: não é Teredo/túnel. Existe um adaptador Teredo no sistema como fallback automático do Windows, mas está inativo, o tráfego real sai pelo endereço global nativo. Confirmado via Test-NetConnection e validado contra a captura do Wireshark.

Ferramenta de captura: Wireshark, captura na interface vEthernet (Hyper-V) 2

Topologia adotada - cenário A, usado como base a infra da atividade 3:

```
[c1] --net-c1-- [r1] --net-transit-- [r2] --net-c2-- [c2]
```

net-c1: IPv4 192.0.2.0/24 (RFC 5737 TEST-NET-1) + IPv6 fd00:1234:5678:1::/64

net-transit (r1↔r2): IPv4 198.51.100.0/24 (TEST-NET-2) + IPv6 fd00:1234:5678:ffff::/64

net-c2: IPv4 203.0.113.0/24 (TEST-NET-3) + IPv6 fd00:1234:5678:2::/64

c1 = 192.0.2.11, r1 = 192.0.2.1 / 198.51.100.1, r2 = 198.51.100.2 / 203.0.113.1, c2 = 203.0.113.12

Por que dois roteadores em série: a Atividade 03 tinha c1 e c2 no mesmo segmento L2, com apenas um roteador anunciando e prefixo via SLAAC, sem rotear tráfego entre eles. Para provocar um ICMP Time Exceeded de verdade, item 4 do roteiro, é necessário que o pacote atravesse pelo menos um roteador que precise decrementar o TTL/Hop Limit e descartar. Por isso a topologia foi redesenhada com duas sub-redes de cliente conectadas por uma rede de trânsito entre dois roteadores dual-stack.

2. Cenário A: ICMP em rede local controlada

Endereços IPv4 adicionados conforme sugestão do roteiro (192.0.2.11 para c1); c2 ficou em 203.0.113.12 por estar em outra sub-rede, o roteiro sugeria 192.0.2.12, mas entendo que isso pressupõe o mesmo segmento L2 da Atividade 03, a adaptação foi necessária pelo redesenho com roteamento.

Endereços IPv6, SLAAC foi herdado do mecanismo da Atividade 03: fd00:1234:5678:1::/64 anunciado por r1 (lado de c1), fd00:1234:5678:2::/64 anunciado por r2 (lado de c2).

Captura iniciada antes dos testes: tcpdump -i any -n -vv 'icmp or icmp6' em r1.

Tráfego gerado e resultado:

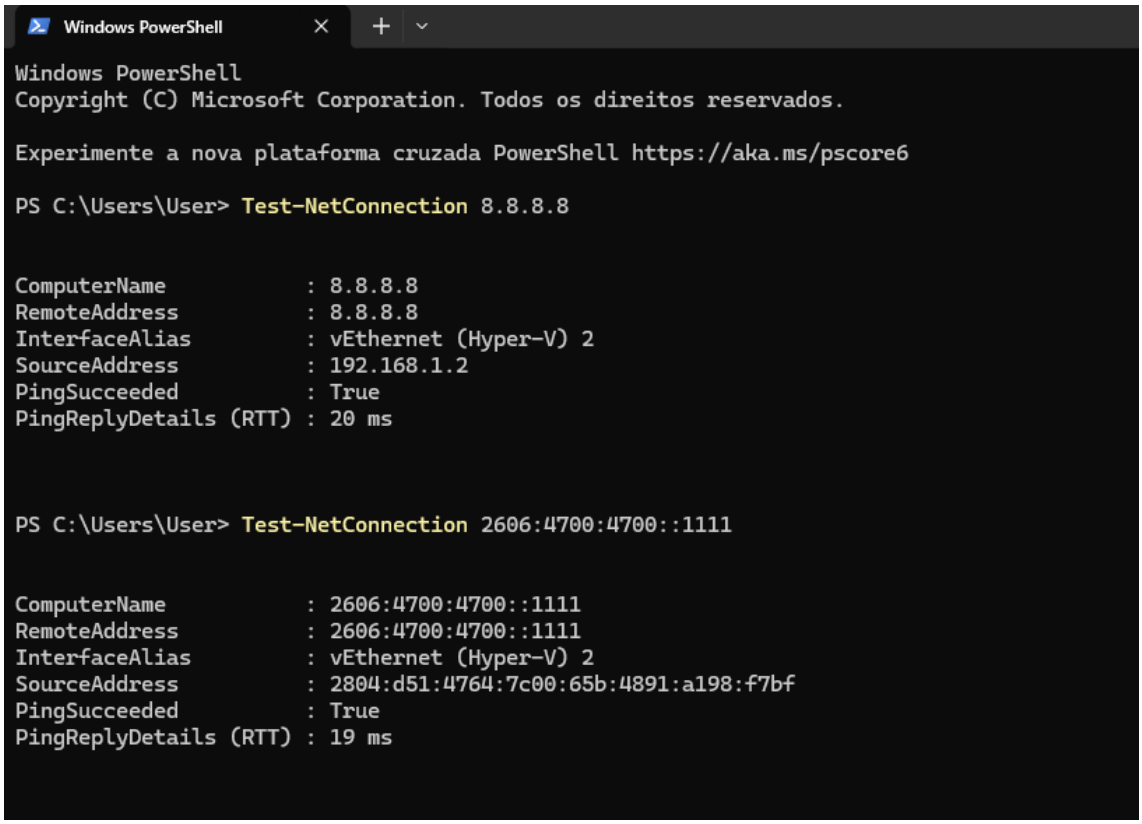
ping -4 -c 5 203.0.113.12 (c1 -> c2): 5/5 recebidos, TTL de resposta 62 (64 - 2 saltos, confirma a passagem por r1 e r2)

ping -6 -c 5 <IPv6 de c2>: 5/5 recebidos, mesmo padrão (hop limit 62)

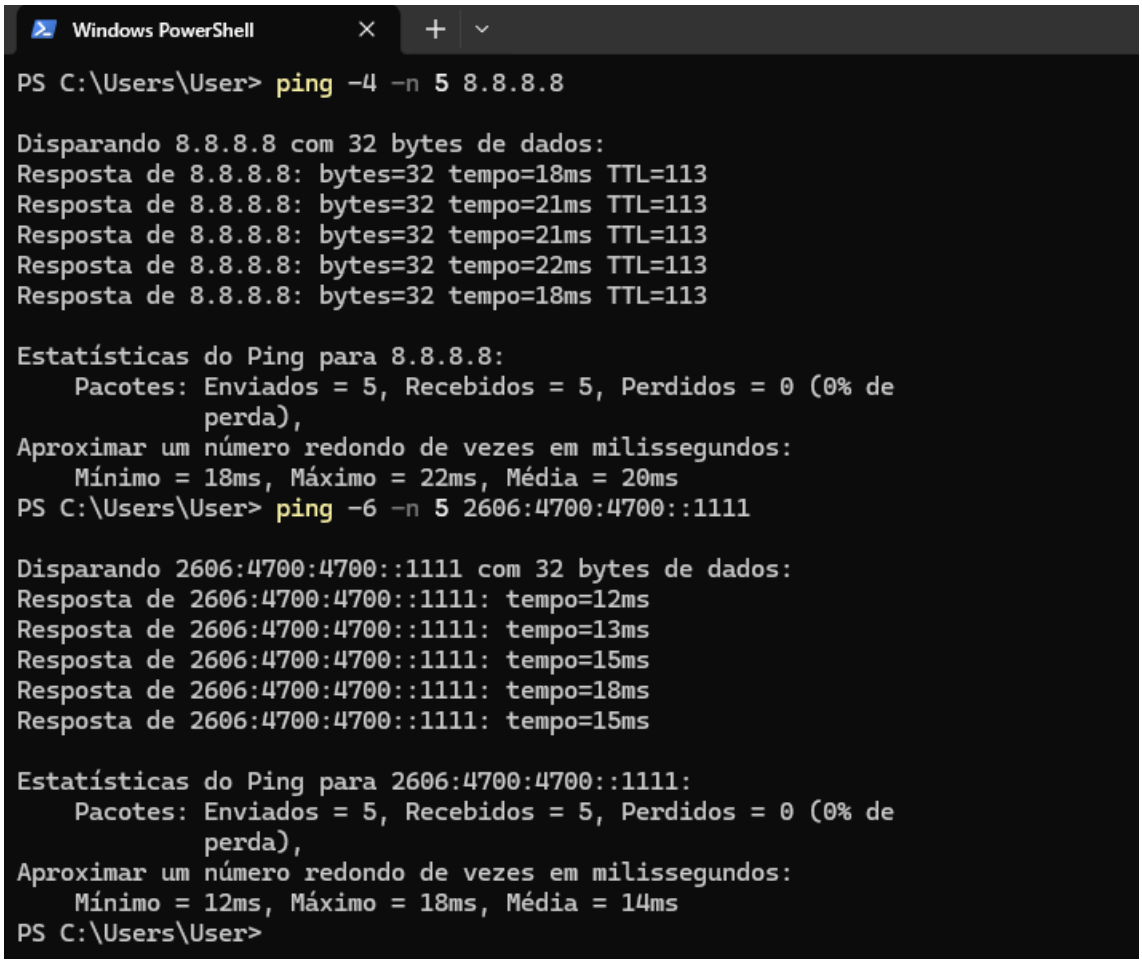
Bug de infraestrutura encontrado e corrigido: o IPv6 falhava entre c1 e c2 até que fosse identificado que c1 tinha duas rotas padrão IPv6 com a mesma métrica, uma real (via RA do r1) e uma "fantasma" do próprio IPAM do Docker. Removida a rota do Docker manualmente.

3. Cenário B: ICMP em rede externa (Internet)

Executado direto no host, no Windows, interface vEthernet (Hyper-V) 2, com captura no Wireshark.



Verificação inicial de conectividade com Test-NetConnection contra os dois alvos do Cenário B, IPv4 (8.8.8.8, RTT 20 ms) e IPv6 nativo (2606:4700:4700::1111, RTT 19 ms), ambos com sucesso.



Identificação da rota usada (via Test-NetConnection -InformationLevel Detailed):

- IPv4 -> 8.8.8.8: InterfaceAlias: vEthernet (Hyper-V) 2, SourceAddress: 192.168.1.2, NextHop: 192.168.1.254
- IPv6 -> 2606:4700:4700::1111 (Cloudflare): mesma interface, SourceAddress: 2804:d51:4764:7c00:65b:4891:a198:f7bf (endereço global real), NextHop: fe80::1

Tráfego gerado (ping -4 -n 5 / ping -6 -n 5, sintaxe do Windows):

- IPv4 -> 8.8.8.8: 5/5 recebidos, RTT mín/média/máx = 18/20/22 ms, TTL de resposta 113
- IPv6 -> Cloudflare: 5/5 recebidos, RTT mín/média/máx = 12/14/18 ms (destino diferente do teste IPv4, Google vs. Cloudflare, então a diferença de latência não pode ser atribuída só ao protocolo)

Frames de referência:

- IPv4: frame 7627 (request) / 7629 (reply), id=0x0001, seq=37
- IPv6: frame 8239 (request) / 8243 (reply), id=0x0001, seq=91

4. Erro provocado: TTL / Hop Limit expirado

Testado nos dois cenários, com -t (Linux/iputils, dentro dos containers) ou -i (no Windows).

Cenário A (ambiente controlado, containers):

TTL/Hop Limit	IPv4	IPv6
1	Time Exceeded de 192.0.2.1 (r1)	Time exceeded de fd00:1234:5678:1::1 (r1)
2	Time Exceeded de 198.51.100.2 (r2)	Time exceeded de fd00:1234:5678:ffff::2 (r2)
3	Sucesso, chega em c2	Sucesso, chega em c2

Cenário B (Internet real)

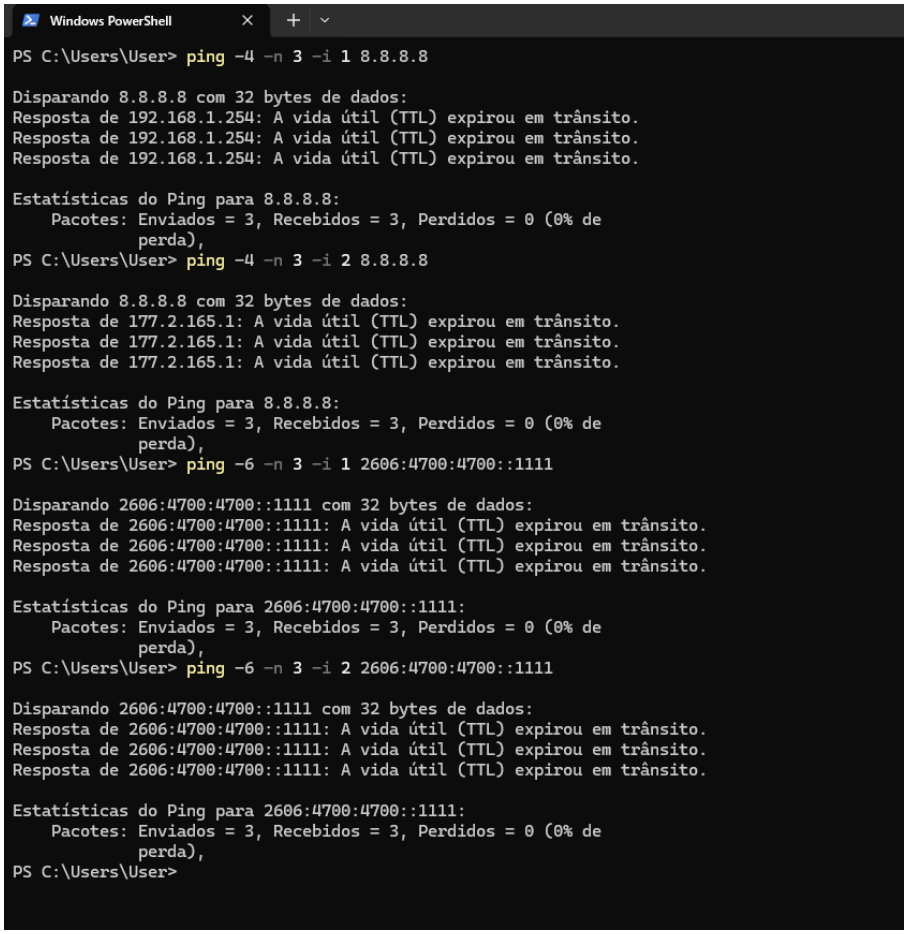
TTL/Hop Limit	IPv4	IPv6
1	Time Exceeded de 192.168.1.254 (roteador local)	Time exceeded de 2804:d51:4764:7c00::1 (roteador local)
2	Time Exceeded de 177.2.165.1 (1º salto do provedor)	Time exceeded de 2804:d50:1:3000::1 (2º salto do provedor)
3	Nenhuma resposta	Nenhuma resposta

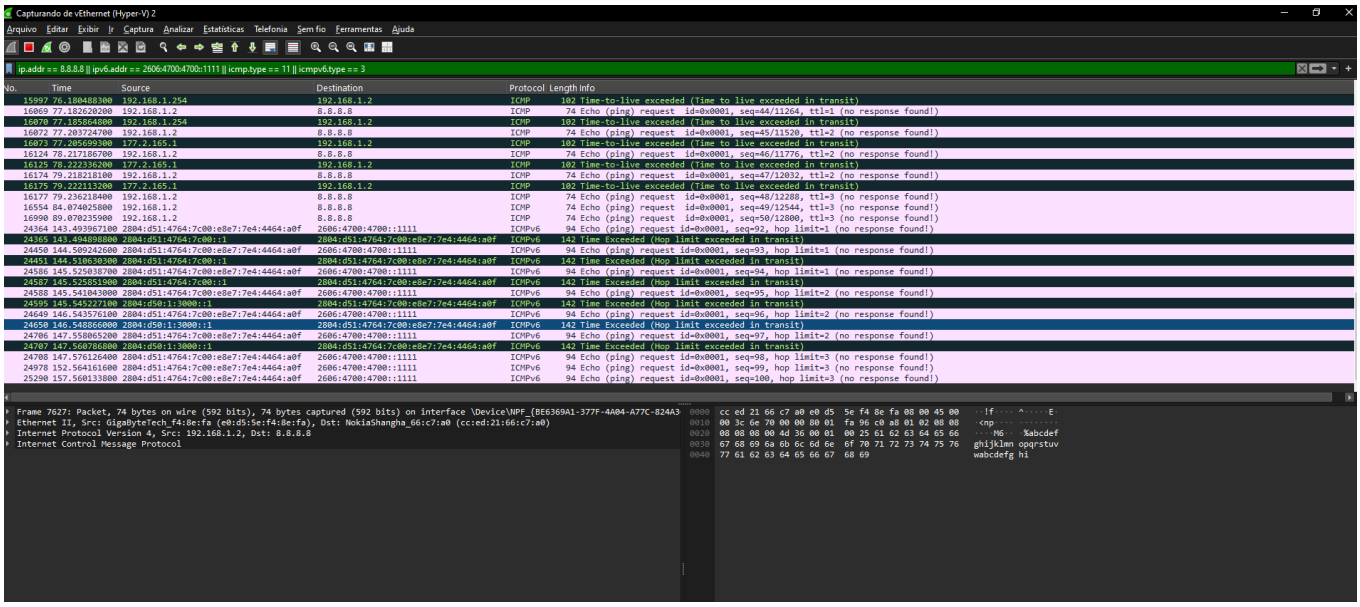
OBS: TTL/Hop Limit do item 3 foi confirmado na captura do Wireshark.

Achado interessante, um bug de exibição do ping.exe do Windows em IPv6: nos dois casos de erro (Hop Limit=1 e =2), o texto impresso pelo ping mostrou incorretamente "Resposta de 2606:4700:4700::1111" (o próprio destino Cloudflare) em vez do endereço real do roteador que gerou o erro. Isso

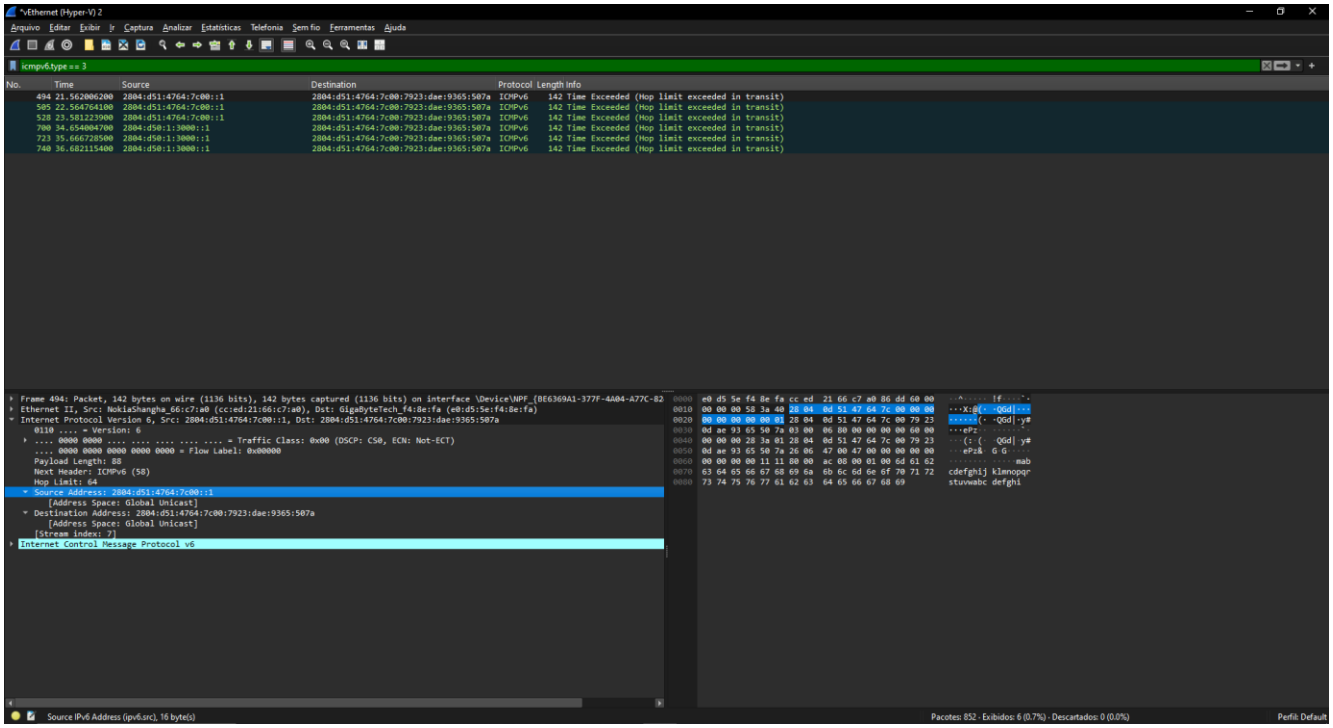
só foi descoberto e corrigido comparando com a captura real no Wireshark, que mostrou as origens corretas (2804:d51:4764:7c00::1 e 2804:d50:1:3000::1). O IPv4 não apresentou esse problema, o ping.exe mostrou os endereços corretos desde o início.

Achado do salto 3: tanto em IPv4 quanto IPv6, o terceiro salto real da Internet não gerou nenhum erro visível, nem no texto do ping, nem na captura do Wireshark. Interpretação: algum equipamento nesse ponto da rede do provedor tem política de não gerar (ou não deixar passar) mensagens de erro ICMP.

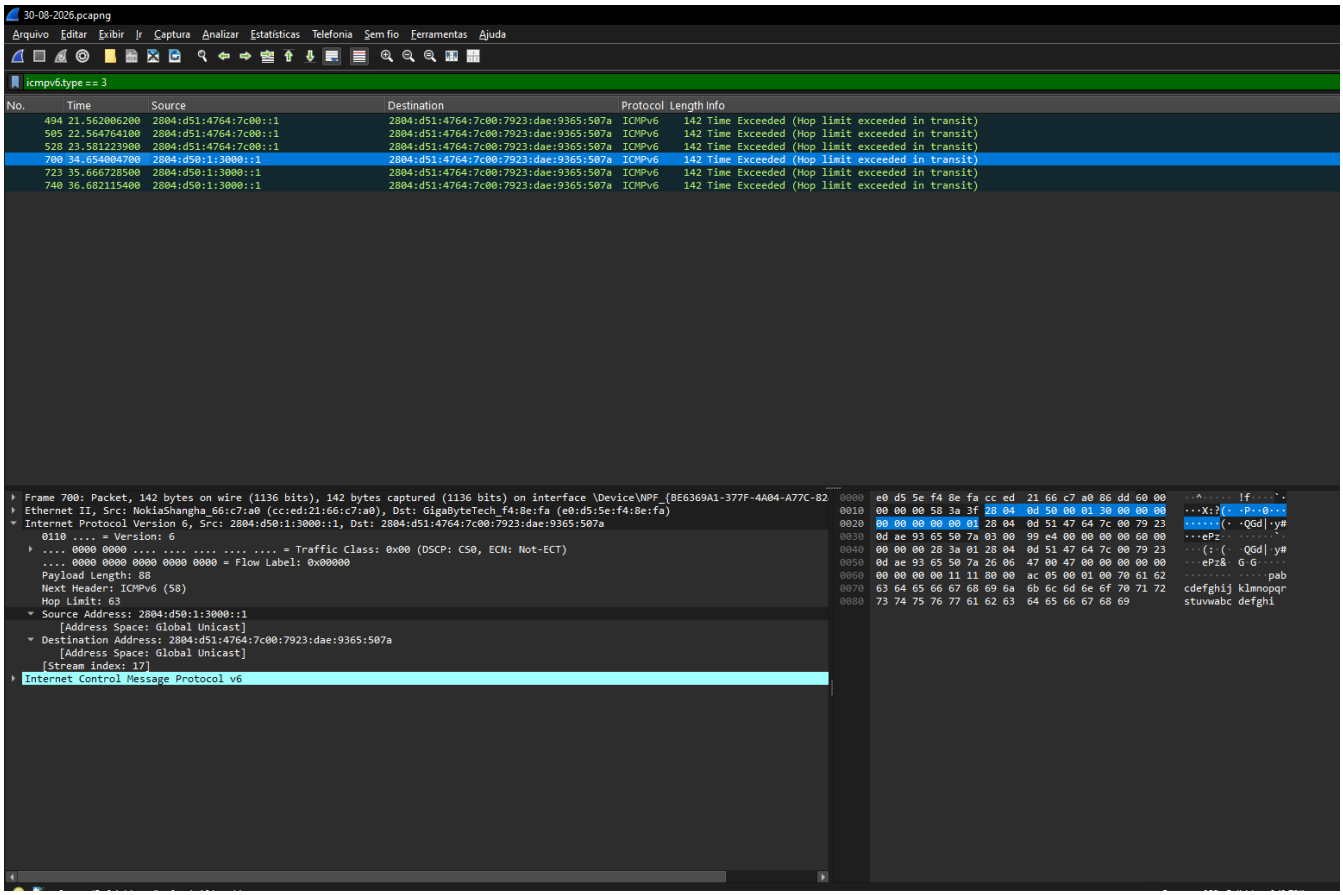




Escalonamento de TTL/Hop Limit contra a Internet real. captura filtrada por destino e por tipo de erro ICMP/ICMPv6. Nos dois protocolos, o padrão é idêntico: erro no salto 1, erro no salto 2, silêncio total no salto 3.



Detalhe do pacote ICMPv6 Time Exceeded capturado no Wireshark durante o teste de Hop Limit=1 contra a Internet real. O campo "Source Address" (2804:d51:4764:7c00::1) mostra o endereço real do roteador de casa que gerou o erro, diferente do endereço exibido pelo ping.exe na tela (2606:4700:4700::1111, o próprio destino Cloudflare).



Detalhe do pacote ICMPv6 Time Exceeded referente ao segundo salto (Hop Limit=2). O campo "Source Address" (2804:d50:1:3000::1) mostra o roteador do provedor, responsável real pelo erro nesse salto, confirmando individualmente o mesmo padrão observado em IPv4 (192.168.1.254 no salto 1, 177.2.165.1 no salto 2).

5. Análise dos pacotes capturados

1) Quais tipos e códigos de ICMPv4 foram observados? Para cada um, associe a mensagem a uma ação realizada no experimento.

- Type 8, Code 0 (Echo Request): os pings enviados, tanto locais (c1->c2, Cenário A) quanto contra 8.8.8.8 (Cenário B).
- Type 0, Code 0 (Echo Reply): respostas correspondentes a cada Echo Request.
- Type 11, Code 0 (Time Exceeded - TTL exceeded in transit): gerado nos testes de TTL baixo (1 e 2), tanto pelos roteadores do próprio laboratório (r1, r2) quanto por equipamentos reais (roteador de casa, primeiro salto do provedor).

2) Quais tipos e códigos ICMPv6 foram observados? Diferencie mensagens de diagnóstico (Echo) das relacionadas ao funcionamento do IPv6.

- Diagnóstico: Type 128 (Echo Request) e Type 129 (Echo Reply), Code 0, mesma função do ping em IPv4.

- Funcionamento do IPv6: Type 133/134 (Router Solicitation/Advertisement), descoberta de roteador e prefixo para SLAAC; Type 135/136 (Neighbor Solicitation/Advertisement), resolução de endereço IPv6→MAC (equivalente ao ARP).

- Erro: Type 3, Code 0 (Time Exceeded, hop limit exceeded in transit), mesmo papel do Type 11 do IPv4, mas como mensagem própria do ICMPv6.

3) Escolha um Echo Request e o Echo Reply correspondente em IPv4. Identifique Type, Code, Checksum, Identifier e Sequence Number e explique como você determinou que os dois pacotes pertencem à mesma troca.

Frames 7627 (request) / 7629 (reply), Cenário B, 192.168.1.2 -> 8.8.8.8.

- Request: Type 8, Code 0, Identifier 0x0001, Sequence 37.
- Reply: Type 0, Code 0, mesmo Identifier 0x0001, mesma Sequence 37.
- Checksum: valores diferentes entre request e reply, conferir o valor exato de cada um diretamente no painel de detalhes do Wireshark para os dois frames. Determinação de que pertencem à mesma troca: Identifier e Sequence Number idênticos nos dois pacotes (é exatamente para isso que esses campos existem, por que como o ICMP não tem portas como TCP/UDP, é assim que o SO correlaciona request e reply), endereços IP invertidos (origem vira destino e vice-versa), o Wireshark confirma automaticamente a relação (anota "reply in 7629" no request e "request in 7627" no reply).

4) Repita a análise para IPv6. Quais campos são equivalentes e quais diferenças você observou?

Frames 8239 (request) / 8243 (reply), Cenário B, endereço global do host -> 2606:4700:4700::1111.

- Request: Type 128, Code 0, Identifier 0x0001, Sequence 91.

- Reply: Type 129, Code 0, mesmo Identifier e Sequence. Campos equivalentes: Type/Code, Checksum, Identifier e Sequence Number existem nos dois protocolos com a mesma função.

Diferença observada: em ICMPv6, o cálculo do Checksum inclui um pseudo-cabeçalho com os endereços IPv6 de origem/destino, igual ao TCP/UDP, enquanto em ICMPv4 o checksum cobre só a própria mensagem ICMP, por isso o valor não é diretamente comparável entre os dois protocolos mesmo com payload parecido.

5) Compare os cabeçalhos IP dos pacotes IPv4 e IPv6 usados no ping. Como a diferença de tamanho entre os cabeçalhos influencia o tamanho total dos frames quando a carga útil do ICMP é semelhante?

Cabeçalho IPv4 mínimo (sem opções): 20 bytes. Cabeçalho IPv6: sempre 40 bytes (tamanho fixo, sem opções no cabeçalho base). Evidência direta da própria captura: com o mesmo payload de 32 bytes (padrão do ping do Windows) e cabeçalho ICMP/ICMPv6 de 8 bytes, os frames IPv4 ficaram em 74 bytes e os IPv6 em 94 bytes, uma diferença de exatamente 20 bytes, batendo precisamente com a diferença entre os dois cabeçalhos (14 Ethernet + 20 IPv4 + 8 ICMP + 32 payload = 74; 14 Ethernet + 40 IPv6 + 8 ICMPv6 + 32 payload = 94).

6) Compare a latência do cenário local com a da Internet. Calcule a média, o mínimo e o máximo a partir dos seus testes e explique as causas prováveis das diferenças.

- Local (Cenário A, c1->c2 via r1/r2): IPv4 min/avg/max = 0.091/0.133/0.215 ms; IPv6 = 0.077/0.122/0.280 ms.
- Internet (Cenário B): IPv4 (8.8.8.8) min/avg/max = 18/20/22 ms; IPv6 (Cloudflare) min/avg/max = 12/14/18 ms.

A diferença é de duas ordens de grandeza (frações de milissegundo vs. dezenas de milissegundos).

Causas prováveis: no Cenário A não existe meio físico real, os pacotes trafegam por bridges de software dentro do mesmo kernel (essencialmente

cópia de memória), enquanto no Cenário B o tráfego atravessa múltiplos links físicos reais, vários roteadores fazendo decisão de encaminhamento de verdade, e distância física real até os servidores do Google/Cloudflare (atraso de propagação, que não existe num laboratório local).

7) No experimento de TTL/Hop Limit, qual equipamento gerou a mensagem de erro? Como você verificou isso?

Depende do valor de TTL/Hop Limit usado, cada valor expõe o equipamento que fica exatamente a essa distância da origem: no Cenário A, r1 (TTL/Hop Limit=1) e r2 (=2). No Cenário B, o roteador doméstico (=1) e o primeiro salto real do provedor (=2).

Verificação: conferindo o endereço de origem do próprio pacote de erro capturado, não o texto impresso pelo comando ping, verificação que se mostrou essencial, já que descobrimos que o ping.exe do Windows exibe incorretamente essa origem para erros ICMPv6 (ver pergunta 10).

8) O pacote ICMP de erro contém informação do pacote que causou o problema? Mostre quais informações permitiriam ao host de origem associar o erro ao tráfego enviado.

Sim. A mensagem Time Exceeded inclui, embutido depois do seu próprio cabeçalho, o cabeçalho IP/IPv6 original completo e o início do pacote que causou o erro. No exemplo capturado (Hop Limit=1, Cenário B), dentro do próprio pacote de erro aparece: o cabeçalho IPv6 original (Hop Limit: 1, origem = o próprio host, destino = 2606:4700:4700::1111) e, dentro dele, o ICMPv6 Echo Request original. É exatamente esse conteúdo embutido, em especial o Identifier e o Sequence Number do Echo Request original, que permite ao host de origem casar o erro recebido com o pacote específico que ele mesmo enviou.

9) Se você capturou Neighbor Solicitation/Advertisement (IPv6) ou ARP (IPv4), explique por que aparecem antes/durante a comunicação.

NS/NA (tipos 135/136) resolvem endereço IPv6 -> MAC, o equivalente direto do ARP em IPv4, difere de RS/RA (133/134), que é quem descobre o roteador e o prefixo para SLAAC. Antes de c1 conseguir enviar um Echo Request para c2, ele já sabe o IPv6 de c2, mas não sabe o MAC, sem o MAC,

o quadro Ethernet não pode ser montado, então o pacote não sai fisicamente. Por isso o NS/NA aparece antes do primeiro ICMP/ICMPv6 de cada nova troca: é um pré-requisito de camada 2, não de camada de rede (o host já tem IP nos dois lados; falta resolver o endereço físico). A diferença fundamental para o IPv4/ARP (conforme o professor explicou em aula) é que o IPv6 não usa broadcast, o NS é enviado via multicast solicited-node (ff02::1:ffXX:XXXX), evitando incomodar todos os hosts da rede como o broadcast do ARP faz.

10) Houve algum resultado diferente da sua hipótese inicial? Explique o que a captura revelou e qual hipótese você reformularia.

Hipótese inicial: esperava que o texto do ping no Windows mostrasse corretamente o endereço do equipamento que gerou cada erro de TTL/Hop Limit, já que isso é o comportamento documentado do ICMP.

O que a captura revelou: em IPv4 isso realmente aconteceu (o texto bateu com a origem real capturada). Mas em IPv6, nos dois casos de erro (Hop Limit 1 e 2), o ping.exe exibiu incorretamente o endereço do destino final (Cloudflare) como se fosse ele quem tivesse gerado o erro, logicamente impossível, já que um pacote com Hop Limit 1 ou 2 não pode ter alcançado o destino. Só percebi a diferença comparando com a captura no Wireshark, onde as origens reais apareceram corretamente (2804:d51:4764:7c00::1 no salto 1, 2804:d50:1:3000::1 no salto 2).

Hipótese reformulada: acredito que o ping.exe do Windows tem um problema específico de exibição para mensagens de erro ICMPv6 (não é um problema de rede nem de captura), isso reforça a lição de não confiar no texto de uma ferramenta sem validar contra a captura bruta dos pacotes.

6. Comparação a partir das evidências

Aspecto	**ICMPv4 observado**	**ICMPv6 observado**	**Evidência / interpretação**
Echo Request / Reply	Type 8 (request) / Type 0 (reply), Code 0	Type 128 (request) / Type 129 (reply), Code 0	Frames 7627/7629 (v4) e 8239/8243 (v6), ambos com Identifier 0x0001, protocolos numeram os tipos de forma

			totalmente diferente, mas a lógica de correlação é idêntica
Mensagem de erro	Type 11, Code 0 (Time Exceeded - TTL exceeded in transit)	Type 3, Code 0 (Time Exceeded - hop limit exceeded in transit)	TTL/Hop Limit=1 -> 192.168.1.254 (v4) / 2804:d51:4764:7c00::1 (v6); =2 -> 177.2.165.1 (v4) / 2804:d50:1:3000::1 (v6), mesmo mecanismo, números de tipo diferentes
Type e Code	Números pequenos e historicamente fragmentados (0, 8, 11, 3...)	Números reorganizados numa faixa mais coerente (128-137 para diagnóstico/NDP)	Observado diretamente nos filtros usados (icmp.type, icmpv6.type) durante a captura
Checksum	Cobre apenas a mensagem ICMP	Cobre a mensagem ICMPv6 + pseudo-cabeçalho com endereços IPv6 de origem/destino (RFC 4443)	Valores de checksum não comparáveis diretamente entre os dois protocolos mesmo com payload idêntico, por causa desse pseudo-cabeçalho extra no IPv6
Tamanho do pacote	74 bytes (14 Ethernet + 20 IP + 8 ICMP + 32 payload)	94 bytes (14 Ethernet + 40 IPv6 + 8 ICMPv6 + 32 payload)	Mesmo payload de 32 bytes nos dois; diferença de exatamente 20 bytes = diferença entre os cabeçalhos IPv4 (20B) e IPv6 (40B)
Resolução de vizinhança	ARP - protocolo separado, fora do IP, usa broadcast Ethernet	NS/NA - dentro do próprio ICMPv6 (Type 135/136), usa multicast solicited-node	NS/NA capturados antes de cada nova troca entre c1 e c2 e host e roteador; IPv6 não tem broadcast, não poderia reaproveitar o mecanismo do ARP
TTL / Hop Limit	Decrementado por roteador; ao chegar a 0, gera Type 11	Mesmo mecanismo, mesmo nome de campo, gera Type 3	Escalonamento 1 -> 2 -> 3 idêntico nos dois protocolos, incluindo o mesmo "silêncio" real no terceiro salto da Internet em ambos
Função observada no experimento	ICMP cobre só diagnóstico (Echo) e erro (Time Exceeded); resolução de vizinhança (ARP) e configuração de endereço (DHCP) são protocolos separados	ICMPv6 concentra diagnóstico, erro, e as funções de infraestrutura do protocolo (RA/NS para SLAAC e resolução de vizinhança)	Essa foi a diferença estrutural mais evidente do experimento: no IPv4 o ICMP é auxiliar; no IPv6 ele é parte da própria "espinha dorsal" do protocolo (sem ICMPv6, não existe autoconfiguração nem resolução de endereço)

Obs.: o ping.exe do Windows não exibe o Hop Limit de resposta para IPv6 (só mostra tempo=Xms), diferente do IPv4 onde exibe TTL=X. Mais uma inconsistência de exibição do comando no Windows, na mesma linha do bug de endereço de origem discutido na seção 4.

7. Uso de Inteligência Artificial

Ferramenta utilizada: Claude (Anthropic), via Claude Code — a mesma da Atividade 03, usada em todas as etapas (planejamento, infraestrutura, execução, escrita).

Ao planejar a atividade 4, pensei e usei a recomendação do professor, em seguir como uma continuação da Atividade 03, cheguei propondo adicionar "outro roteador pra IPv4, se fizer sentido". O Claude apontou algo que eu não tinha percebido: na topologia da Atividade 03, c1 e c2 estavam no mesmo segmento L2, e nesse caso um ping -t 1 simplesmente funcionaria normal, hosts finais não decrementam TTL, só roteadores decrementam. Ou seja, não era "se fizer sentido", era necessário ter pelo menos um roteador de verdade no meio pra sequer conseguir reproduzir o experimento de TTL pedido no roteiro. Isso mudou a arquitetura toda: viramos pra uma cadeia de dois roteadores dual-stack (c1 - r1 - r2 - c2), com sub-redes diferentes de cada lado.

Na implementação, apareceu um bug: o IPv4 funcionou de primeira, mas o IPv6 dava 100% de perda entre c1 e c2. Investigando junto, achamos que o c1 tinha duas rotas padrão IPv6 com a mesma métricam uma real (via RA do roteador) e uma "fantasma" do próprio IPAM do Docker (mesmo subnet decoy que usamos na Atividade 03 pra provar SLAAC). O kernel escolhia a errada. Corrigido removendo manualmente a rota do Docker.

No Cenário B, contra a Internet, o achado mais importante veio comparando o texto do ping.exe do Windows com a captura real do Wireshark: pra IPv4 os endereços batiam certinho, mas pra IPv6 o ping.exe mostrava o endereço de destino (Cloudflare) como se fosse ele quem tivesse gerado o erro de Hop Limit, logicamente impossível. Só desconfiamos porque o Claude notou a inconsistência lógica (um pacote que expirou em 1-2 saltos não pode ter chegado ao destino final) e sugeriu conferir a captura bruta em vez de confiar no texto do comando. A captura confirmou: as origens reais eram roteadores intermediários (2804:d51:4764:7c00::1 e 2804:d50:1:3000::1), nada a ver com o Cloudflare.

Pergunta feita à IA antes de executar o experimento, e como validei a resposta: perguntei se fazia sentido simplesmente reaproveitar a topologia da

Atividade 03 (c1/c2 no mesmo link) pro experimento de TTL desta atividade. A resposta foi que não funcionaria, pelo motivo explicado acima. Validei isso na prática: construímos a topologia com os dois roteadores e confirmamos, rodando de verdade, que TTL=1 expira exatamente em r1 e TTL=2 em r2, se a hipótese do Claude estivesse errada e um único link bastasse, isso não teria acontecido.

Sugestão que precisei confirmar/corrigir após observar minha rede: o Claude sugeriu inicialmente investigar o bug do ping.exe como possível problema da minha rede (rota incorreta, firewall etc.). Só depois de abrir a captura e comparar pacote por pacote é que ficou claro que a rede estava correta, o problema era de exibição do próprio ping.exe, não da infraestrutura. Sem essa checagem, o relatório poderia ter concluído erroneamente que havia algo quebrado no meu roteador ou provedor, e que também não fazia sentido por que a conexão está funcionando corretamente.

Evidência concreta usada para validar a conclusão final: a captura no Wireshark filtrada por icmpv6.type == 3, mostrando o endereço de origem real dos pacotes de erro (frame 494 para o salto 1; 700 para o salto 2), completamente diferente do que o ping.exe reportava na tela.